一芯多模
面向资源受限 Zynq 平台的多网络视觉推理与软硬件协同系统

摘要

在低成本边缘视觉设备上，更换视觉任务通常意味着更换模型，甚至重新设计专用硬件。本作品"一芯多模"在 EES-331 开发板的 Zynq-7020 芯片（集成 ARM 处理器 PS 与 FPGA 可编程逻辑 PL）上探索另一条路线：把网络计算从"固定电路"改为"可装载程序"，让多个神经网络共享硬件执行资源并在线切换。

为此，作品借鉴 Overlay Processing Unit（OPU）的组织思想[13-14]，自主实现编译器、RTL、驱动和验证链。编译器把网络整体或经适配的指定子图编译为程序卷，自定义指令集描述卷积与图算子，硬件执行层以 64 个双路 INT8 处理单元、共享量化尾部和页式调度执行任务。当前 YOLOv8n 整网任务划分为 9 页，后续优化围绕任务编译和可加载执行上下文展开。

系统保留网络原始结构，已在统一摄像头—HDMI 视频底座上实现同一 BIT 下三网同任务、三网三任务以及 NanoDet 纯 PS 与 YOLOv8n PS+PL 双后端在线切换，无需重新下载位流；同时完成 YOLOv8n 网络主体纯 PL 推理。第五类 OPU 硬件任务换载处于仿真阶段。固定输入实验中，整网 9 页计算由约 19.15 秒降至约 6.24 秒（50MHz，约 3.1 倍），80MHz 稳定点为 4.27 秒；连续整网推理段均值约 11.02 秒，混合推理 488 帧中位数约 2.50 秒；计算模块同范围综合 LUT、触发器分别下降 25.0%、50.8%。各数字均对应特定版本与统计条件，不作跨条件合并。

作品同时建立四层验证和大模型协作审计流程，为后续编译优化、流水深化与资源复用提供可复现基础。

第一部分 作品概述

1.1 功能与特性

作品实现"一套硬件、多种网络、在线切换"的嵌入式视觉推理系统，具备五项功能：①多网络切换——单份位流内切换 NanoDet、YOLOv8n、YOLO11n 三个网络，完成同一手势任务，或物体检测、手势、姿态三种任务；②网络主体硬件推理——通过 OPU 编译器、自定义指令集与硬件执行层，把 YOLOv8n 网络主体映射为纯 FPGA 计算；③双后端在线切换——同一手势任务下，用拨动开关在"纯软件推理"与"软硬件混合推理"两种后端间在线切换；④完整视觉链路——FPGA 完成摄像头采集、按各网络输入约定的缩放填边预处理、检测框与文字的画面合成及 HDMI 显示；⑤版本可追溯——模型、程序卷、位流与输入约定按版本绑定并留存验证记录。YOLOv8n 与 YOLO11n 的硬件推理在线换载正在仿真验证；机械臂联动仅保留历史 PS 软件推理链验证成果，当前各硬件推理形态按对应位流独立验收，未集成路径以虚线表示。

1.2 应用领域

本系统面向低成本、任务轮换的边缘视觉原型，适用于实验教学、交互展示和非高速巡检验证。同一套相机、显示与控制接口可复用：任务简单时选软件推理，需要硬件加速时把网络整体或子图交给 FPGA，介于两者之间时采用软硬件混合，从而减少重复搭建外围系统的工作。

对需要经常更换模型的原型开发，OPU 提供从网络模型到硬件任务程序的统一映射路径，其价值在于降低新网络、新切分方案的适配成本，而不是把某一个网络推到极限速度。当前验证主要在室内摄像头与固定测试输入上完成，秒级推理适用于演示与方案研究，尚不代表工业实时部署能力。

1.3 主要技术特点

①一套体系多种用法：编译器、自定义指令集与共享硬件执行模块构成 Overlay 式三层计算体系，已先后承载纯 FPGA 整网与混合推理检测头。②保留

网络结构：不剪枝、不为单一网络改造数据通路，以共享计算阵列和可生成任务程序替代专用电路，为换网保留统一基础。③软硬件协同分工：PS 承担模型服务、部分推理、输入输出处理与调度，PL 承担视频采集显示、相应预处理与硬件计算，分工点可按任务调整。④全程可追溯验证：每项性能对应特定版本与统计条件，模型、程序卷、位流与输入约定按版本绑定。

1.4 主要性能指标

表 1-1 主要性能指标

项目	实测值与范围
软件多任务	单份位流三网络三任务在线切换
双后端切换	纯 PS 与 PS+PL 拨码在线切换，11 次
固定输入计算	50MHz：19.153→6.239 秒；80MHz：4.267 秒
连续整网	83 帧日志，推理段均值 11.016 秒
混合推理	488 帧，中位数 2.497 秒
资源优化	LUT −25.0%，FF −50.8%，控制集 −46.2%

1.5 主要创新点

①一体三层体系：统一视频底座、Overlay 式计算体系与在线换载相结合，纯软件、纯 FPGA、软硬件混合三种推理后端在同一块板上实测。②编译期统一映射：以固定数据通路加任务描述字承载原始网络结构，已验证整网与检测头子图两级映射。③混合切点与双后端：检测头卸载到 PL、主干留在 PS，两种后端共用一份位流、拨码在线切换。④把大模型协作中的失败经验沉淀为可复用的部署、仿真与审计技能包。

1.6 设计流程

先打通摄像头到 HDMI 的视频底座并建立软件推理参考；再实现 OPU 编译器、指令层、计算阵列与图算子，按处理单元、模块、整网三级逐级验证；依据实测剖析优化访存、预取、流水与资源映射；随后集成连续视频与软硬件混

合推理，实现双后端在线切换；最后按版本绑定资产、整理实测结果与复现说明。在研能力单独标注，不计入已完成成果。

第二部分 系统组成及功能说明

本部分先给出系统全景（2.1：组成、五类工程总览、术语约定），再按官方模板依次介绍硬件（2.2）、软件（2.3），然后把五类工程共同依赖的 OPU 计算体系集中讲清（2.4），随后逐形态展开（2.5），最后给出优化过程与前后对比（2.6）。

2.1 整体介绍

2.1.1 选题背景与要解决的问题

本作品参加 AMD 自主选题高级组，围绕软硬件协同与器件整体利用展开[1]。选用 Zynq-7020，是因为它把 ARM 处理器（PS）与 FPGA（PL）放进同一芯片、共享同一 DDR 内存：PS 适合运行成熟的软件推理框架，PL 适合构建并行计算电路，两者的分工边界可以由设计者按任务重新划分。

项目要解决的核心问题是：在 Zynq-7020 有限的片上资源与内存带宽下，如何建立一条可复用的"网络→硬件"映射路径，使多个网络共享同一套硬件，并让 PS 与 PL 的计算分工可以调整。专用加速器能为单个模型做到很高效率，但每换一个网络通常要重新设计数据通路；ncnn 代表适合嵌入式处理器的软件路线[5]；FINN 展示了量化网络与可重构数据流结合的路线[10]。本作品选择保留网络原始结构，用 OPU 体系探索通用映射，不以单一网络的极限时延为唯一目标。系统按"可观察的完整视觉链路→可逐位校验的硬件计算→多种部署形态"推进；跨网络的硬件换载目前仍在仿真阶段。

2.1.2 系统组成与数据流

系统由四部分组成。PC 离线工具：训练、模型导出、量化（把网络权重与激活转成整数运算）和 OPU 程序卷生成；PS 软件：运行 PYNQ Linux 与 ncnn 推

理服务，负责模型选择、输入准备、硬件发令、后处理与日志；PL 硬件：持续运行视频采集、DMA 搬运与显示叠加，并按位流配置承担预处理、检测头或整网计算；外围设备：OV5640 摄像头、DDR3 内存与 HDMI 输出，蓝牙与成品机械臂用于历史执行端实验。DDR 保存视频帧、权重与中间特征，是 PS 与 PL 交换数据的位置。

两点说明：显示链按视频节拍持续刷新，识别结果在推理完成时更新，因此 HDMI 视频帧率不等于模型处理速度；五类工程共享同一视频底座，但各自的网络切分、输入约定与验证范围分别定义。



图 2-1 系统总体架构与数据流

2.1.3 五种工程形态总览

整个项目按五类工程形态推进，它们是全文的骨架：表 2-1 给出每类的 PS/PL 分工与当前状态，2.5 节逐形态展开。

表 2-1 五种工程形态总览

工程	PS 职责	PL 职责	当前状态
一、三网同任务	三网络完整推理与解码	采集、统一缩放填边、显示、模型选择接口	已完成板上验证
二、三网三任务	检测、手势、姿态网络与解码	视频与 OSD 合成；手势通道使用 PL 预处理	已完成板上验证
三、纯 PL 整网	输入变换、运行控制、解码	网络主体计算、采集与显示	已完成固定输入与连续运行验证
四、双后端切换	NanoDet 整网，或 YOLOv8n 主干/颈部及解码	YOLOv8n 六检测分支、视频与拨码接口	已完成在线切换验证
五、多网换载（在研）	模型选择与程序装载	复用 OPU 硬件执行层	YOLO11n 仿真进行中

五类工程逐级递进：工程一、二先证明"软件层多模型＋统一视频底座"；工程三证明 OPU 能承载完整网络；工程四证明 OPU 可以只承载子图、并与纯软件后端共存在线切换；工程五在此基础上把第二个网络接入同一硬件。它们共同依赖的 OPU 体系在 2.4 节集中介绍。

2.1.4 术语与阅读约定

本作品包含一批工程术语，全部在表 2-2 集中定义；正文首次出现处不再重复解释，外部通用缩写在首现处随文注释。

表 2-2 关键术语约定

术语	白话解释
OPU 体系	本作品借鉴 Overlay Processing Unit 的组织思想形成的"网络→硬件"映射体系，由编译器、自定义指令集和共享硬件执行模块组成，详见 2.4
程序卷	编译器输出的硬件"可执行程序包"：由指令、描述符、量化参数与地址布局组成的二进制文件，部署时整体写入内存、按版本归档（"卷"取卷宗之意）

术语	白话解释
页 / 九页	任务程序的分段执行单位，类似书本按页阅读：当前 YOLOv8n 整网任务划分为 9 页，硬件一页一页执行，页与页之间由 PS 确认完成并绑定下一页
门铃	PS 与硬件执行层之间的通知寄存器：PS 写入即"按铃"（绑定某页、启动计算），硬件把完成与错误状态写回供 PS 读取
输入合同	对输入图像的完整预处理约定（尺寸、等比缩放与填边、颜色通道顺序、归一化公式、类别编号含义）；训练、PS 软件、PL 硬件三份实现必须逐项一致，任何一项漂移都会"合法地"输出错误结果
固定输入实验	用一组固定图像（而非摄像头）作输入，硬件执行完整网络并用片上周期计数器计时的对比实验；输入固定、输出可逐位比对，各优化版本可在完全相同条件下比较
逐位比对	硬件每张输出特征图与 PC 端整数参考逐字节比对；如"21/21"表示该网络末端的 21 张输出特征图全部一致
整数参考	在 PC 上用与硬件完全相同的整数运算（同一量化参数、同一舍入规则）实现的软件参考模型
工程形态	一类可独立上板运行的部署配置（网络切分＋位流＋软件＋验证范围），本文共五种，见表 2-1
位流（BIT） / HWH	BIT 是决定 PL 电路结构的 FPGA 配置文件；HWH 是配套的寄存器与地址描述文件，PYNQ 据此驱动硬件
OSD / letterbox	OSD＝屏幕叠加显示（On-Screen Display），在视频画面上实时画检测框、文字与骨架；letterbox＝等比缩放加补边，把任意尺寸画面装入网络输入尺寸
统计条件（口径）	一项性能数字所对应的版本、输入、精度、频率与计时起止点；不同统计条件的结果不互相换算或合并

2.1.5 设计演进主线

工程按"先软件、再整网、后混合"推进：先积累参考，再扩大硬件计算范围。早期由 PS 逐算子组织数据，数据整理与打包成为主要开销；随后改为 OPU 描述符与页调度，把跨端控制收敛为"装载—发令—确认"；整网验证完成后，又

把 OPU 承载范围收缩为检测头子图，形成混合推理与双后端切换。演进围绕网络切分、任务程序生成和相应硬件裁剪展开，不同工程形态采用各自 BIT；但在同一工程配置内，多网络与多任务在线切换无须更换 BIT（过程与数据见 2.6）。

2.2 硬件系统介绍

2.2.1 硬件整体介绍

平台为 EES-331 开发板，核心器件 xc7z020clg484-1：双核 ARM Cortex-A9、53,200 个 LUT、220 个 DSP48E1、140 个 36Kb 等效 BRAM[2]。外围连接 OV5640 摄像头、DDR3 内存与 ADV7511 HDMI 发送器；开发工具为 Vivado 2025.2 与 PYNQ 3.0.1；位流、HWH、程序卷与软件按版本绑定交付。

自主设计部分包括 OPU 计算阵列、量化尾部、图算子、页调度、预处理逻辑、OSD 与主机驱动；处理器系统、VDMA、AXI 互连、视频时序与存储 IP 属于基础组件。移植到其他板卡需重新核对管脚、电源、摄像头时钟、DDR 地址、设备树与时序约束。

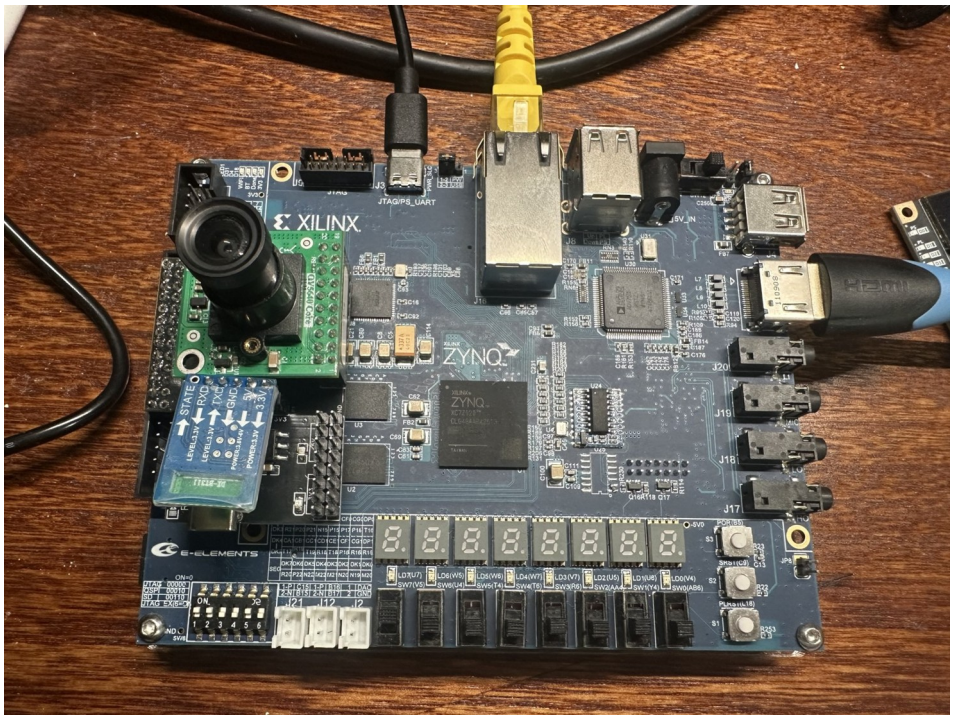


图 2-1a EES-331 开发板与摄像头实物

2.2.2 机械设计介绍

机械工作主要包括摄像头、开发板和外设的安装连接及扩展基座集成。开发板与机械臂为现成器件，系统设计贡献集中在电气连接、供电组织和视觉到执行端的接口实验，不将机械臂机构与伺服系统列为自主研发。

现有照片展示板卡和工作台搭建情况，基座尺寸、紧固位置和走线尚未形成完整制造图纸。历史 PS 软件推理链已完成 13 次触发、13 次动作、0 次失败的机械臂联动验证（见 3.2.1），该记录仅归属于对应历史配置。当前硬件推理形态中，纯 PL 整网因片上资源接近上限暂未集成该路径，其他链按实际配置独立验收；架构图以虚线表示相应形态的未集成路径，补集成后仍需独立联调。

2.2.3 电路各模块介绍

视频采集与显示链。采集侧通过 SCCB 总线配置摄像头，接收像素时钟、行有效、帧同步与像素数据，转换为 AXI4-Stream 视频流；VDMA（视频直接存储器访问 IP）的写通道把视频流写入 DDR 帧缓存，读通道按显示节奏读回。AXI4-Stream 用 TVALID/TREADY 握手，跨时钟域使用相应视频 IP 及同步结构。显示链实际串接为：VDMA 读通道→视频输出转换→检测框叠加→文字叠加（OSD）→HDMI 发送器，视频时序由 VTC 模块提供。PS 只写入类别、坐标和置信度等参数，PL 在像素扫描时完成合成，避免处理器逐像素修改整帧；三任务版另有物体标签和姿态骨架逻辑。

预处理模块（仅存在于对应配置）：三网手势工程把 640×480 图像用整数双线性缩放为 320×240，再上下各填 40 行灰值 114，输出 320×320×3 字节的 RGB 图像（HWC 排列）；PL 输出为无符号字节，PS 继续转为 CHW 排列与浮点，并执行归一化。该逐位一致性结论仅适用于此版本、此算法及测试样本。

表 2-3 硬件模块清单（计算类模块细节见 2.4）

模块	关键输入	关键输出	验证关注点
摄像头采集	pclk、vsync、href、像素总线	视频流及帧行边界	帧尺寸、颜色顺序、跨时钟
VDMA	视频流、帧地址、步长	DDR 帧与显示读流	状态寄存器、错误位、帧缓存归属
PL 预处理	源地址、目标地址、控制寄存器	307200 字节 RGB 图像	320 行写回、边界像素、填充值
GEMM 阵列	输入/权重块、量化参数	INT8 特征或累加结果	位宽、饱和、舍入和双路解包
图算子与页调度	描述符、页表、门铃	内存结果与完成状态	地址范围、依赖顺序、错误复位
OSD	检测框、类别、分数、字库	合成 RGB 与同步信号	索引、缩放、帧边界更新

2.3 软件系统介绍

2.3.1 软件整体介绍

软件分两端。PC 侧形成训练、导出、校准、图映射和参考生成流程；板端包含模型服务、PYNQ 控制层、连续运行器、后处理、显示接口与日志记录。部署资产有四类，共同构成一个可核对的版本单元：模型文件（网络参数）、程序卷（硬件执行计划）、HWH（寄存器与地址描述）、BIT（硬件电路配置文件）。

软件模式下，多个模型装入常驻服务，由模型选择状态切换对应输入与解码；硬件模式下，先准备权重、特征和控制表，检查连续物理内存与缓存一致性，再触发计算。两条路径都把识别结果转换为统一的"框、类别、分数"表示，以复用同一显示接口。

2.3.2 PS 软件模块与逐帧流程

连续整网（工程三）的逐帧流程为：取得帧快照→执行输入变换→保存输入与哈希→调用硬件运行器→解码输出→写日志→更新显示参数；发生错误时先记录当前帧状态，再按策略清空或保持显示，避免沿用旧结果。

混合路径（工程四）逐帧流程为：PS 向 ncnn 模型服务取得三个尺度的特征图→按硬件要求量化并写入连续物理内存区（Linux CMA）→清理处理器缓存（flush）→写门铃启动 PL→PL 完成 6 个检测分支（3 个尺度×回归/分类两类）→PS 使缓存失效（invalidate）后读回→依次完成反量化、DFL 解码（把回归通道输出的分布还原成框坐标）、sigmoid、NMS（非极大值抑制，去除同一物体的重叠候选框）和坐标逆变换。



图 2-2 软件逐帧处理流程与关键输入输出

表 2-4 软件分层与失败定位

层级/入口	主要输入变量	主要输出变量	失败定位
帧获取与输入变换	frame、模型槽位、尺寸/颜色参数	网络张量、input_sha	帧快照与预处理参考

层级/入口	主要输入变量	主要输出变量	失败定位
模型服务	输入张量、模型选择	预测或三尺度特征	模型哈希、输出节点及 shape
硬件运行器	CMA 地址、权重、程序卷	输出面、状态、周期计数	flush/invalidate、错误码、页序
解码与筛选	blob、scale、阈值	detections	类别映射、DFL/NMS 及坐标
记录与显示	seq、耗时、rc、detections	JSONL、OSD 参数	新旧帧归属与显示阈值

2.3.3 训练与模型适配任务

训练目标由通用准确率逐步转向板端实际困难：左右手势混淆、拇指类背景误触和摄像头域差异。数据处理固定类别顺序、左右语义、缩放填边及归一化；导出后检查数值和类别，避免离线标签与板端含义不一致。

表 2-5 训练版本与结果

模型与版本	任务性质	主要结果或用途
YOLOv8n v1	七手势首版训练	69 张测试集 mAP50 为 0.730
YOLOv8n v2	数据迭代训练	228 张测试集 0.972，173 张留出集 0.983
YOLOv8n 定向重训	左右语义与学习率修复	难点类 Right 置信度提升至 0.839
YOLO11n 重训与量化	手势及姿态模型适配	用于三网手势、三任务姿态和后续换载
NanoDet v2→v3	负样本定向重训	Thumbs Down 负样本置信度 P95 由 0.4459 降至 0.0583
YOLOv8n COCO	硬件验证与量化校准	用于整网任务映射和整数参考验证
NanoDet COCO 预训练	三任务物体检测	未针对本摄像头重训，保留跨域局限说明

表中精度来自不同数据集和版本，不跨集计算提升幅度。NanoDet 的 INT8 候选因部分指标回退未纳入部署，保留浮点模型；YOLOv8n 与 YOLO11n 采用各自验收过的候选。训练曲线取自 10 月 23 日定向重训的 25 个 epoch 记录，最佳权重位于第 5 个 epoch。

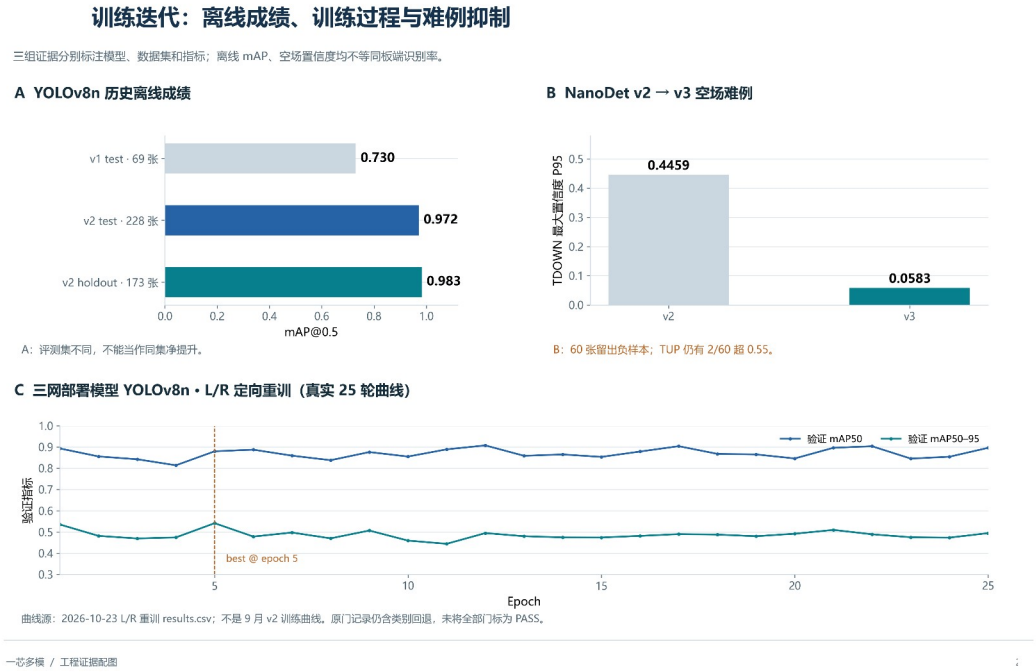


图 2-3 训练版本与学习曲线

2.3.4 PS 推理优化与分网络适配

软件路径由通用实现迁移到面向 ARM 优化的 ncnn 推理框架，利用 NEON 向量指令（ARM 的单指令多数据扩展）加速卷积，并让模型常驻内存以减少重复加载。历史 OpenCV dnn 约 2.47 秒与对应 ncnn 约 0.814～0.970 秒的对比说明该阶段软件栈收益（提速 2.55～3.03 倍）；各网络现役时延分别统计。

表 2-6 分网络软件优化与结果

网络	共享优化	分网络适配与结果
NanoDet	ARM NEON、模型常驻、PL 预处理衔接	轻量交互模型，约 0.78 秒；INT8 候选未采用，保留浮点部署
YOLOv8n	ncnn 转换链、常驻服务、哈希绑定	完整 PS 推理约 1.05 秒；抽取三尺度特征用于 PS+PL 检测头卸载

网络	共享优化	分网络适配与结果
YOLO11n	同框架模型服务与输入合同管理	支持手势及姿态任务；三任务姿态拆分引擎约 2.9～3.9 秒

单线程口径用于保证与采集、显示和网络服务稳定共存；双核处理器上的多线程收益不作为承诺指标。

2.4 OPU 体系：从网络到硬件的统一映射

本节集中介绍五类工程共享的计算体系：先讲 OPU 是什么（2.4.1），再自上而下讲编译器（2.4.2）、指令层（2.4.3）、硬件执行层（2.4.4 计算阵列、2.4.5 图算子与页调度），最后说明 PS/PL 分工（2.4.6）与整网到子图的一次真实复用（2.4.7）。

2.4.1 OPU 是什么

本作品借鉴 Overlay Processing Unit（OPU）的组织思想[13-14]，将网络计算划分为模型编译、自定义任务描述接口和共享硬件执行模块，形成 Overlay 式三层计算体系。该术语用于描述架构方法，不表示本作品属于某一公开 OPU 处理器的兼容实现，也不表示已经具备任意网络在线换载能力。编译器、RTL、驱动和验证链均为本工程自主实现。

三部分分工：编译器把网络转换为程序卷和任务流（2.4.2）；自定义指令集是编译器与硬件之间的统一描述接口（2.4.3）；硬件执行层完成调度、计算和结果写回（2.4.4、2.4.5）。当前已验证 YOLOv8n 整网与检测头子图两级映射；不同工程形态按功能裁剪并采用各自 BIT，但同一工程内多网络和多任务在线切换无须更换 BIT。新增 OPU 硬件任务换载仍需检查算子覆盖、形状、量化规则和地址生命周期。

2.4.2 OPU 编译器

编译器读取网络结构、权重、量化参数和张量形状，确定算子覆盖范围、数据依赖和软硬件切分，随后分配 DDR 与片上缓冲区，生成卷积控制信息、图

算子描述符、页控制、布局文件及参考输出。编译期检查维度、地址范围和依赖关系；运行期检查页完成和最终输出。当前编译流程支持 YOLOv8n 整网和六检测分支任务生成；后续将加强算子融合、网络图优化和调度效率，使新网络、新切分方案更快映射到硬件。

2.4.3 OPU 自定义指令层

指令层是编译器与硬件执行层之间的接口，共五类组成（表 2-7）。以卷积控制为例，它由 ISS256、GEO106、WLAY168 三组独立控制字组成，分别描述调度、几何和权重布局——三组字各自输出，并非一条连续 530 位总线。

表 2-7 指令层组成

组成	主要作用	当前实现边界
CONV	描述卷积分块、K 段、形状、地址和权重布局	三组独立控制字及配套量化参数
PPU DESC	描述图算子的输入输出、形状、量化与分段	固定 128 字节描述符（32 个 32 位字）
PPP	在卷积边界触发对应图算子物理槽	32 位控制字，与图算子操作码位宽不同
VIEW	以地址别名复用父张量，减少数据搬移	编译期元数据，不占运行时指令槽
门铃	页绑定、启动、完成与错误状态控制	绑定与启动命令各占一个寄存器写，页按序串行执行

完整位域定义和真实编码实例见随工程提交的指令集字段表（CONV / PPU / PPP / 门铃四份字段表）。

2.4.4 计算阵列与共享量化尾部

卷积计算被映射为分块矩阵乘。物理阵列为 8×8 共 64 个处理单元（PE），每个 PE 含两路 INT8 乘积与累加通道，逻辑上覆盖 8×16 路输出；"128 路"指每拍的逻辑乘加通道数，并非 128 个物理 PE。DSP48E1 打包计算需要处理有符号扩展、交叉位影响与结果拆包[3]（图 2-4b）。

输入与权重经加载逻辑送入阵列，累加结果进入共享的缩放、舍入、饱和及激活尾部；硬件与整数参考使用一致的舍入规则（就近舍入到偶数，RNE）和量化参数。共享尾部以分时调度换取资源节省，也形成吞吐约束；整网速度还受数据加载、边界块和图算子影响。

片上缓存与 FIFO 按访问模式映射到 BRAM（块存储）、LUTRAM（查找表小存储）或移位寄存器。工程曾因描述方式使写 FIFO 被综合器展开为大量触发器，后以综合器可识别的存储模板修正，在不改变网络数学运算的条件下改善布局布线可行性（量化对比见 2.6.6）。

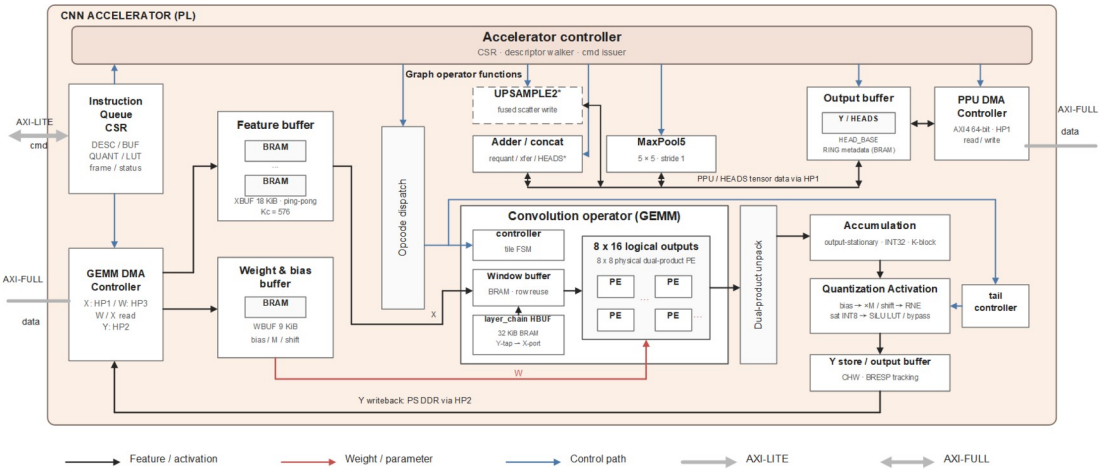


图 2-4a CNN 加速器微架构与数据通路

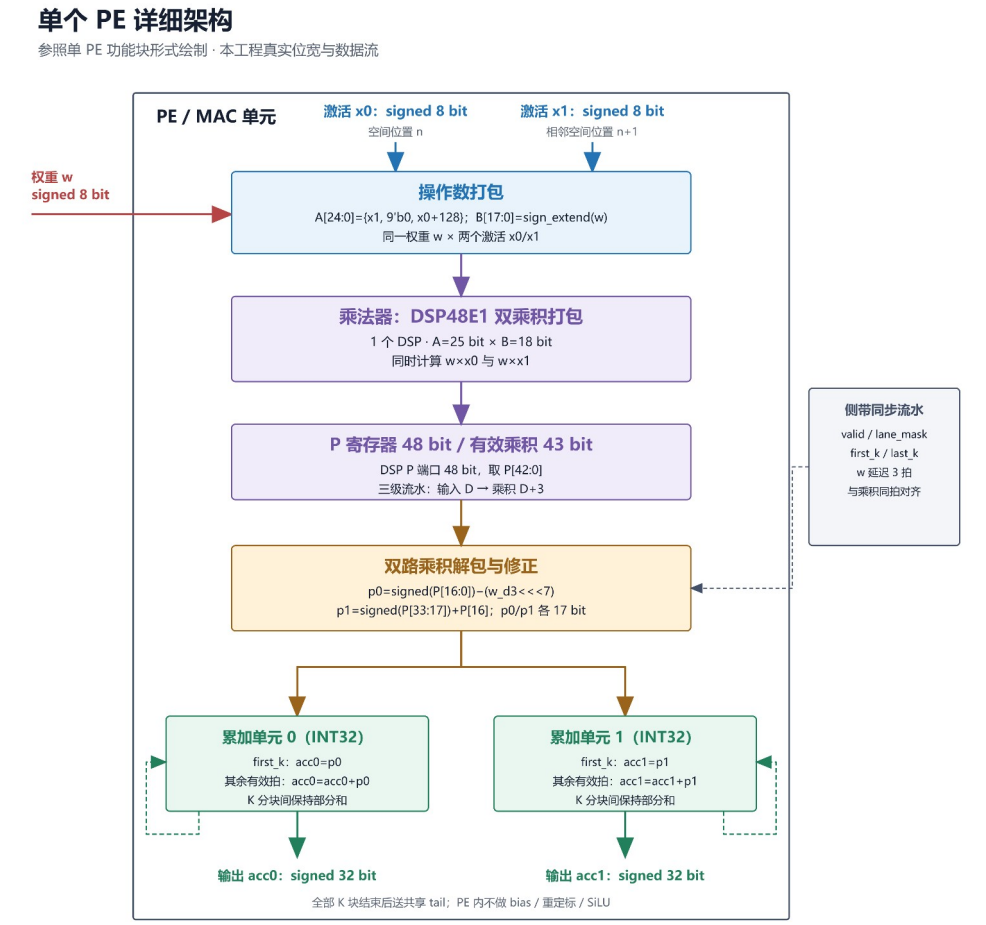


图 2-4b 单处理单元双路乘积与累加结构

单 PE 接收有符号 8 位的权重 w 与两路输入 x0、x1；无符号输入用"加 128 偏置、翻转符号位"的方式进入 DSP。DSP 输出取低 43 位作为有效乘积，再拆包为两路有符号 17 位结果。乘积在输入后第 3 拍有效，双路 INT32 累加结果与完成信号在第 4 拍有效；first_k 信号控制累加起点，K 块之间保持累加状态，无效拍或被屏蔽通道保持原值。偏置、重定标、饱和及 SiLU 激活属于 PE 外的共享尾部（图 2-4b）。

2.4.5 图算子与页调度

网络中除卷积外的结构操作（ADD、CONCAT、MAXPOOL5 等）由图算子硬件承担；上采样（UPSAMPLE2）不设独立模块，由卷积生产者在写回结果时按步长散布完成；检测头聚合（HEADS）在 9 页任务全部结束后，由 PS 单独发一条启动命令执行。

页调度逻辑根据描述符和门铃组织计算窗口，完成页内任务执行、状态反馈与结果写回。连续整网运行时，PS 逐页绑定并启动 9 页任务，9 页结束后启动 HEADS，并检查完成序号和帧完成状态。上述硬件只负责执行已映射的任务；网络结构、地址、形状与量化参数由编译层和指令层提供，硬件本身不解释模型文件。



图 2-5 任务格式与页调度

2.4.6 PS 与 PL 的分工及数据一致性

PS 承担：模型注册与常驻、全部或部分网络的 ncnn 推理、输入变换与量化、程序卷和权重装载、任务发令与状态管理、检测解码与 NMS、显示参数更新、日志记录及外部通信。不同工程中 PS 参与计算的范围不同：软件形态承担完整网络，纯 PL 形态保留输入输出处理，混合形态承担主干与颈部。

PL 由 OPU 硬件执行层与视频外设逻辑组成：前者完成矩阵计算、图算子、页调度和结果写回；后者承担摄像头采集、DDR 视频搬运、相应工程中的缩放填边、检测框和文字合成、HDMI 显示以及拨码选择接口。

接口与一致性：PS 经 AXI-Lite 完成版本检查、参数配置、门铃和状态查询；数据面经四个高性能 AXI 端口（HP0–HP3）直连 DDR——混合版实际连接为 VDMA 到 HP0、输入特征读取 HP1、权重读取 HP3、结果写回 HP2，三网手势版采用另一地址窗口划分，各配置按自身 HWH 核对。PS 在 PL 读数据前执行缓存清理（flush）、PL 写数据后使缓存失效（invalidate），并保证缓冲区只在消费者结束后复用。

2.4.7 从整网到 PS+PL：同源复用与在线切换

纯 PL 整网与 PS+PL 混合形态复用了同源编译器组件、任务描述接口以及 GEMM、量化尾部等计算 IP。转向 PS+PL 后，网络切分和硬件任务重新生成，同时硬件按 heads-only 目标裁剪图算子岛、ROM 和调度结构，形成新的 BIT。这次演进体现了架构方法与工具链复用，而不是把不同工程形态视为同一电路实例。

混合方案切点为 model.15、model.18 和 model.21 三层输出（对应特征尺寸 64×40×40、128×20×20、256×10×10），三尺度量化输入共 179,200 字节。进一步地，双后端工程已在同一 BIT 上实现 NanoDet 纯 PS 与 YOLOv8n PS+PL 在线切换，证明了同一工程内共享硬件资源与任务组织的实际价值；后续统一可加载执行上下文将为更多 OPU 硬件任务换载提供基础。

2.5 五种工程形态逐一说明

本节按表 2-1 顺序逐形态说明：每类给出定位、PS/PL 分工、架构图位与验证边界。

2.5.1 工程一：三网络同任务（PS 推理＋PL 统一预处理）

以七种手势识别为共同任务，NanoDet、YOLOv8n 和 YOLO11n 均由 PS 执行完整推理，PL 完成摄像头采集、统一缩放填边（letterbox）、结果显示与模型选择接口。三个模型经同一常驻服务在线切换，无需重新综合或下载位流。PL 输出无符号字节 RGB 后，由 PS 继续完成归一化、网络推理和解码。该形态

PL 资源占用低（表 3-2 约 20%），历史 PS 软件链已完成机械臂联动验证，PL 侧蓝牙控制路径可复用历史单模型链经验；当前三网集成版的板级机械臂联调未纳入完成成果。

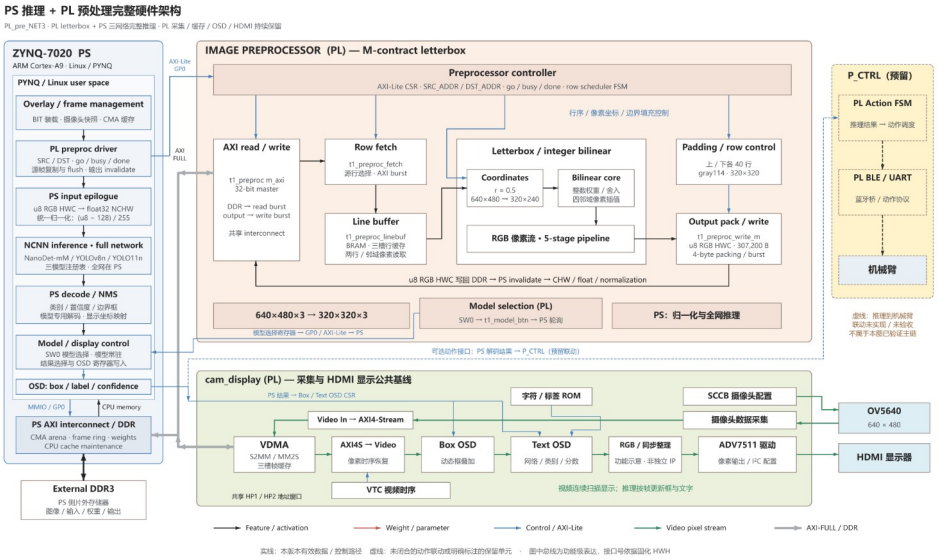


图 2-6 工程一：三网 PS 推理与 PL 统一预处理架构

2.5.2 工程二：三网络三任务（单位流多任务显示）

同一视频位流下，可选 NanoDet COCO 物体检测、YOLOv8n 七手势或 YOLO11n 人体姿态。PS 负责三个网络及对应解码；PL 负责视频及结果的合成显示——物体类别与置信度、手势文字、姿态骨架。仅手势通道使用 PL 预处理，物体检测和姿态由 PS 处理输入。三种模型共用采集与显示底座，经 PL 选择接口在线切换。该工程证明的是软件模型注册表与视频接口复用，不等同于三个网络均已由 OPU 硬件执行（表 2-1 工程三、四才是硬件执行）。

本工程沿用工程一的采集、预处理、显示和模型选择硬件框架，硬件框图与图 2-6 相同。两者的差异主要在 PS 加载的模型与任务解码，以及不同任务的结果显示映射，因此不再重复绘制架构图。

2.5.3 工程三：纯 PL 整网（完整网络的 OPU 硬件执行）

把 YOLOv8n 网络主体映射到 OPU 硬件执行层：PL 完成卷积、图算子和检测头计算；PS 负责输入变换、运行控制、检测解码、结果服务及显示参数更新。

连续版本保留摄像头采集和 HDMI 显示。该形态 PL 资源占用接近上限（表 3-2 为 81.77% LUT），机械臂联动路径因资源不足暂未集成，图 2-7 中以虚线表示；已验证的机械臂闭环在软件推理链的历史路径上（PS 以太网→上位机→蓝牙→机械臂，见 3.3.5）。

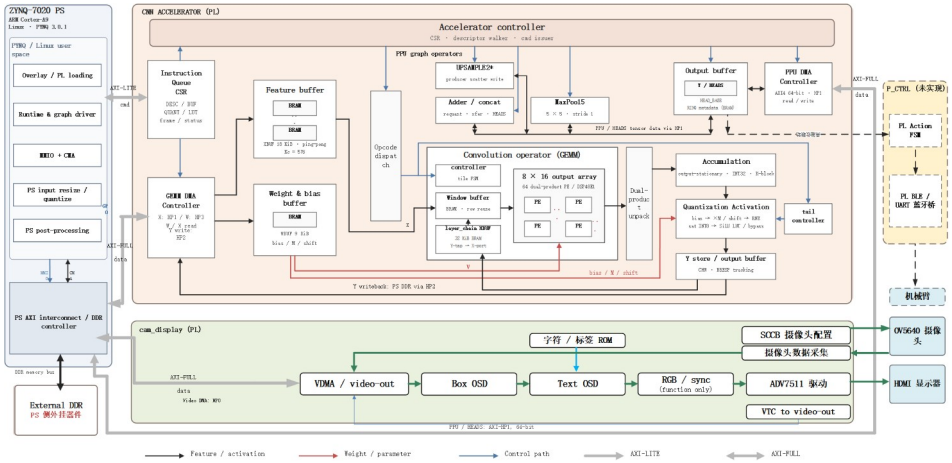


图 2-7 工程三：纯 PL 整网架构

2.5.4 工程四：双后端在线切换（纯 PS 与 PS+PL 共存）

面向同一七手势任务：后端一为 NanoDet 纯 PS 推理；后端二为 YOLOv8n 混合推理（主干/颈部在 PS、6 个检测分支在 PL）。两个后端共用一份位流、视频链和显示模块，PL 拨码开关提供选择电平，PS 在帧边界完成切换。NanoDet 后端复用同一视频底座但不占用 OPU 计算任务；当前混合位流不包含整网图算子岛。归档验证包含 11 次拨码切换和双后端七手势识别；后续可通过更换权重或重训练扩展任务组合。

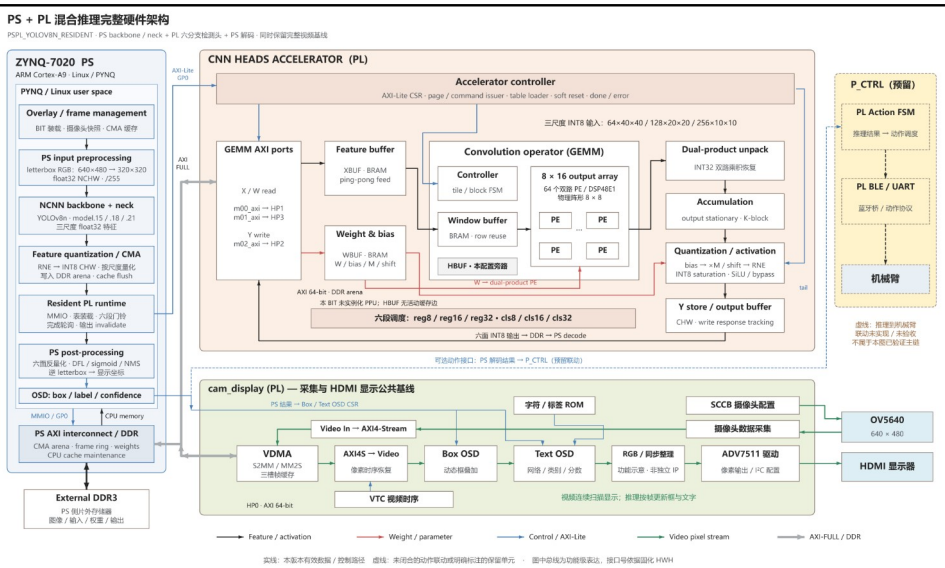


图 2-8 工程四：双后端在线切换架构（混合后端）

2.5.5 工程五（在研）：多网换载，YOLOv8n 与 YOLO11n 复用 OPU

三网同任务、三网三任务及双后端在线切换已经在同一 BIT 上实现，无须重新下载位流。本工程进一步把在线切换扩展到 OPU 硬件任务级换载：正在开展 YOLO11n 的 PS+PL 仿真，目标是在同一 OPU 硬件执行层上支持 YOLOv8n 与 YOLO11n，并与已有纯 PS 后端组成在线切换。换网主要通过编译器适配、统一任务表和控制逻辑调整实现。该工程尚未形成终版位流和板级验收结果，报告不列其推理性能；资源数据仅以双后端基座作设计参考，最终占用以完成后的实现报告为准。

多网络换载实现后，系统还可从不同网络执行同一任务，进一步扩展为不同网络分别执行不同任务。例如在手势识别、物体检测和人体姿态之间切换时，主要工作将集中在模型与程序卷更换、任务选择及结果解码映射，无须为每个任务重新设计硬件，从而显著简化多任务切换流程。受开发时间限制，当前尚未开展不同网络不同任务的在线切换验证。

2.6 优化过程与前后对比

2.6.1 测量条件与统计纪律（立规）

本报告所有性能数字使用三种测量条件之一：①优化后软件参考——同一网络在优化前后软件栈上的推理时间；②固定输入硬件计算——定义见表 2-2，以 9 页任务的片上周期计数计时，每个稳定点以"末端 21 张输出特征图逐位比对全部一致"为验收条件；③连续运行日志——真实摄像头连续运行下按日志统计。

统计纪律（全文适用，后文不再重复）：任何比较都固定模型、输入、精度、频率、位流和计时起止点；不同统计条件的结果不合并为单一加速比，不跨工程排名；行文用约值，表格保留精确实测值。除特别注明外，各数字均为板上实测。

2.6.2 软件栈优化（工程一、二的 PS 基础）

纯 PS 路径由 OpenCV dnn 迁移到 ncnn，利用 ARM NEON 优化卷积，并让多个模型常驻同一服务。对应 NanoDet 由约 2.47 秒降至 0.814～0.970 秒（提速 2.55～3.03 倍）；现役三网络参考时延约 0.78、1.05 和 1.16 秒，三任务姿态拆分引擎另为 2.9～3.9 秒。

2.6.3 纯 PL 两代方案演进

第一代：PS 逐算子组织数据。纯 PL 路线的目标是验证保留原始网络结构时的通用硬件映射，而非针对 YOLOv8n 剪枝或专用结构改造；在 Zynq-7020 资源和单计算核条件下，这一选择限制了可达到的速度，但为多网络复用保留统一基础。第一代由 PS 逐算子组织数据并下发，整网首测 47.62 秒；剖析显示主要时间消耗在 PS 数据整理和打包，硬件计算占比很低。仅优化 PS 数据整理即降至 7.77 秒，但跨端搬数和处理器深度参与仍然存在，因此停止该路线。

第二代：OPU 任务流。把网络转换为任务描述字和页调度程序，PS 只负责装载和发令；配套编译器生成卷积表、图算子描述符和页控制，整网任务与 PC

整数参考逐项核对。层内流水把下一块数据装载与当前块计算重叠，实测代表性卷积层由 53.05 毫秒降至 49.43 毫秒，收益受数据装载主导限制；层间流水链接相邻任务，使前一层尾部与后一层头部交叠，并把检测后处理纳入硬件任务组织。两段并发保留为稳定边界；三段并发出现异常后完整回退。

2.6.4 固定输入计算优化：七个稳定点

固定输入实验下，整网 9 页计算时间的演进见表 2-8（稳定点名称为工程内部代号，含义见表中"主要变化"列）。

表 2-8 固定输入计算：七个稳定点

稳定点	频率/MHz	九页计算/秒	主要变化
基线	50	19.152751	固定输入整网路径
RC 外层	50	14.204047	重排任务与权重使用
双槽预取	50	12.887601	加载侧预取
深度预取	50	10.221841	增大预取窗口
RC-pair	50	6.590057	相邻任务配对
两段并发	50	6.239139	保留已验证并发边界
运行时提频	80	4.267127	同稳定线调整实际时钟

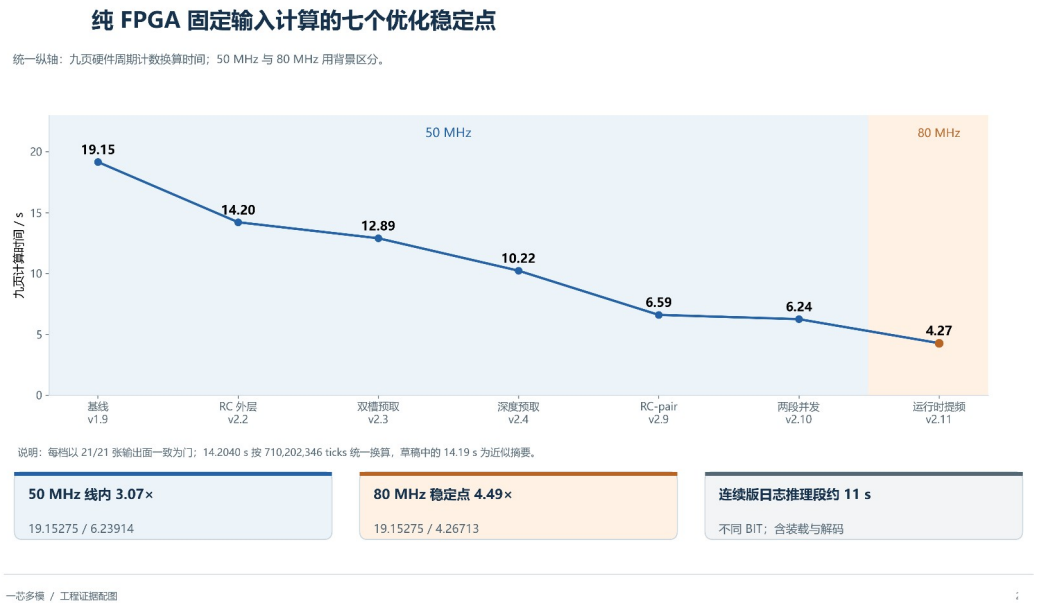


图 2-9 固定输入整网计算时间的七个稳定点

50MHz 线内加速约 3.07 倍，计入 80MHz 稳定点后约 4.49 倍。4.267 秒属较早稳定位流的固定输入计算，不直接套用到集成摄像头和显示的连续版本；早期整机版本约 25 秒，连续运行版 83 条日志的推理段均值 11.016 秒，二者按版本分别报告。资源接近上限后，100MHz 布局尝试未能收敛。为判断物理约束能否在不改变逻辑的条件下改善时序，开展锁布局消融实验，随后进行资源优化。

2.6.5 高资源占用下的锁布局消融实验

实验以 AC1 集成设计为对象，基线 Slice 占用 76.04%、LUT 28,050、DSP 87、BRAM tile 69.5，WNS/WHS 为+0.111/+0.051ns。基线复跑的时序与指纹完全一致，说明比较噪声带为零。实验分别采用软布局锁、资源锁、硬包含和固定布线，保持逻辑功能不变，并检查 DSP 数量及 39 项数据流指纹。

表 2-9 锁布局消融实验分组与结果

分组	约束方式与对象	WNS/WHS（ns）	结果与结论
E0 基线	不增加布局约束	+0.111/+0.051	基线复跑结果一致
T1a	读写双岛软锁，3,522 个单元	+0.455/+0.025	保留，WNS 改善 0.344ns

分组	约束方式与对象	WNS/WHS（ns）	结果与结论
T1b	GEMM 的 77 个 DSP 资源锁	+0.719/+0.020	保留，WNS 改善 0.608ns
T1c	PPU 岛软锁，18,926 个单元	+0.414/+0.013	保留，WNS 改善 0.303ns
T1d	HBUF 的 8 个 BRAM 资源锁	+0.247/+0.020	保留，WNS 改善 0.136ns
T1e	repack 与 glue 软锁，758 个单元	+0.823/+0.011	单组最优，WNS 改善 0.712ns
T2	repack 域硬包含，限制布线出圈	布线失败	171 个引脚不可布，证伪硬包含
T3	13 条 HBUF 主干网固定布线	1 小时 31 分仍未收敛并终止	证伪固定布线，路由自由度受限
T4a/T4b	T1e 参考 checkpoints 增量复跑	+0.823/+0.011	复用率 98.85%，时序零漂移

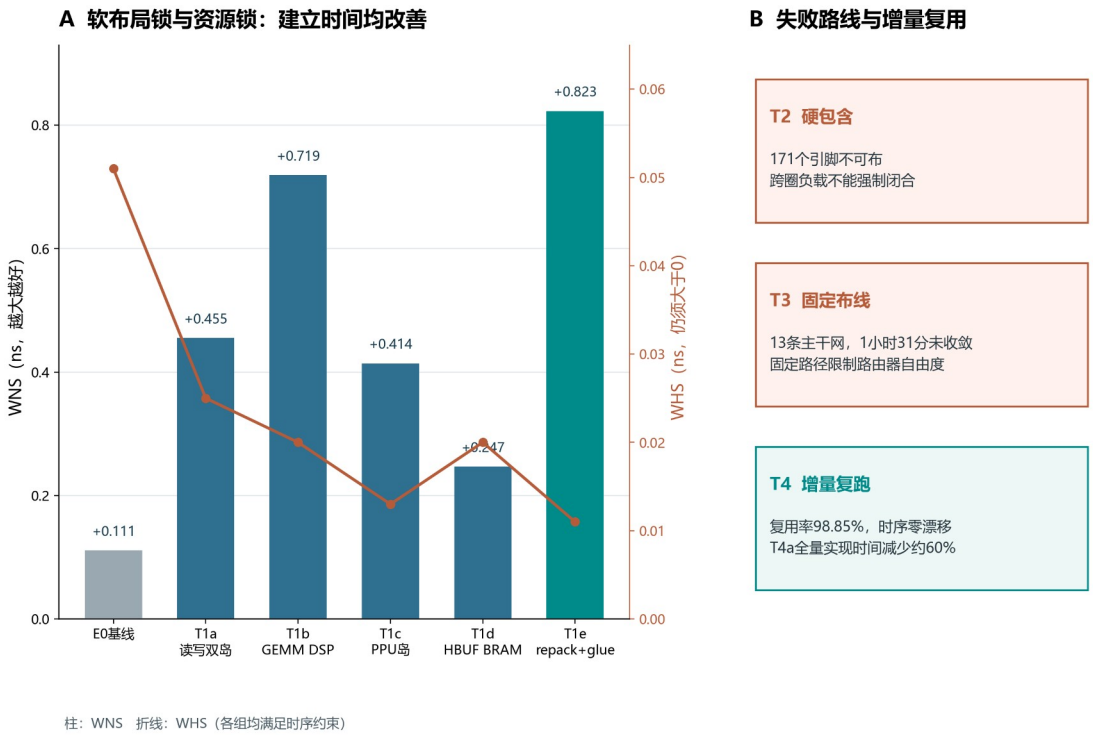


图 2-11 锁布局消融实验结果

软布局锁和资源锁均改善了建立时间，但保持时间裕量有所下降且仍全部为正值，因此应同时检查 WNS 与 WHS，而不能只凭单一指标判断。T1e 为单组最优候选，五种锁的叠加组合未进行验证，不将单组收益直接外推到组合方案。T2 表明跨圈负载不能被硬包含强制闭合，T3 表明在布局已收敛后继续固定主干布线会限制路由器自由度。

增量复跑进一步验证已收敛数据流可以复用：T4a 零改动与 T4b 单 LUT 假改动均保持 98.85%复用率和相同时序，T4a 用时较全量实现减少约 60%。该结果支持在后续优化中保留已收敛区域，而不是每次修改都重新布局整个设计。

2.6.6 资源优化

资源优化五项措施：①写 FIFO 改用 BRAM 实现（原先的描述方式使综合器把它展开成大量触发器）；②延迟链改用 SRL16E 移位寄存器；③行缓存映射为 LUTRAM；④描述符装载逻辑简化；⑤启用 DSP48E1 片间级联输入（PCIN）传递累加值。全部措施不改变网络数学运算。

表 2-10 资源优化前后对比（模块单独综合，同一范围）

指标	优化前	优化后	同范围变化
LUT	37121	27836	−25.0%
触发器	51209	25215	−50.8%
控制集（寄存器控制分组数，越少越利于布局）	1642	883	−46.2%
DSP	90	90	不变
可见 BRAM tile	1	2	增加 1



图 2-10 同范围资源优化前后对比

表中数字来自模块单独综合（OOC，脱离整机只综合该计算模块），BRAM 只统计该范围内可见实例、不含黑盒 IP 内部存储，与全设计结果不可直接相减。集成完整视频与计算的整版设计（内部代号 AC1）布线后为 LUT 28,050、DSP 87、WNS +0.111ns，说明优化后模块在整机中仍可实现。

2.6.7 转向 PS+PL 与驻留优化

纯 PL 连续版本（约 11 秒/帧）尚不足以形成流畅交互，因此把 OPU 承载范围调整为检测头子图，保留 PS 在主干、颈部和结果处理方面的成熟软件能力；该变化复用同源编译器组件、任务描述接口和计算 IP，并按混合目标裁剪硬件形成新 BIT。混合驻留优化把权重与控制表保留在同一进程中，减少逐帧初始化：最终归档 488 帧中位数 2496.7375 毫秒、均值 2554.5529 毫秒，记录 23 次自动恢复；早期 196 帧会话中位数 2492.965 毫秒、恢复 12 次，两组统计分别保留。当前混合路径仍慢于约 1.05 秒的纯 PS YOLOv8n 参考，其主要价值是验证子图卸载、双后端共存以及同一 BIT 在线切换。

第三部分 完成情况及性能参数

本部分汇总已完成工程（工程一至四）的实物成果、量化指标、协作过程与复现入口；工程五为在研，只报状态不报性能。

3.1 整体介绍

已形成四类已完成工程和一类在研工程。各工程拥有独立的位流、模型和运行资产，并非同一份位流同时具备全部功能；模型切换（软件层）、位流切换（重新下载）和 OPU 任务换载（工程五目标）是三种不同的机制，分别定义。

图 3-1 展示开发板、相机、显示和执行端的工作台组成。现有素材尚未形成严格的正面与斜 45°整机配对照片；机械臂照片仅用于说明系统组成，不作为当前链联动证明。



图 3-1 开发板与工作台实物

3.2 工程成果

3.2.1 机械成果

实物集成包括开发板、相机、连接线和成品机械臂的工作台布局。历史单模型控制链保存 13 次触发、13 次完成、0 次失败的记录，以及 79 秒空场无额

外触发的记录，证明对应历史 PS 软件推理配置在该次试验中完成"识别→动作"闭环。纯 PL 整网形态因 PL 资源接近上限暂未集成该路径；其他当前链按实际配置确定验收范围，后续补集成时，推理结果到动作控制模块（P_CTRL）、动作状态机与机械臂的联动仍需重新独立联调。

3.2.2 电路成果

已交付视频底座、PL 预处理、纯 PL 整网计算和混合检测头等配置及其 HWH 与实现报告。三网三任务和双后端工程均保留采集、显示与选择接口，资源数据按具体位流统计。

三任务 OSD（屏幕叠加显示）修复覆盖字体行位段、陈旧 OOC 综合产物、字库索引基址和两倍缩放字符格几何四处：最终姿态标签由 8 像素字符格修正为 16 像素，测试期望由原始像素计数器和规格公式独立推导。归档像素仿真包含 6415 次检查，并以旧版 RTL 触发首个像素失败作为负向对照（证明测试本身有效）。

3.2.3 软件成果

三任务版本在同一视频位流下选择物体检测、七手势和 17 关节姿态。真实 HDMI 照片分别呈现物体类别及分数、手势标签和姿态骨架；姿态显示中的 P92 为置信度信息，并非人数。



图 3-2 三网络三任务真实 HDMI 显示

双后端画面分别呈现 V8PL 与 ND 标识，用于区分两种后端的识别显示。三任务物体检测使用未针对本摄像头重训练的 COCO 模型，室内特定物品仍存在误检或漏检，后续需通过数据采集和量化评估改善域适配。

3.3 特性成果与量化指标

3.3.1 四层验证体系与样本统计（立规）

项目以四层验证构成正确性证据链：①整数参考——检查数值合同（PC 软件模型与硬件逐位一致）；② RTL 仿真——覆盖握手、边界和复位；③实现报告——检查资源与时序；④板上实测——检查真实内存、时钟和连续运行。各层结论按其实际覆盖范围使用。

样本统计纪律（全文适用）：固定输入的 21 张输出特征图逐位比对、历史矩阵块随机测试与连续摄像头记录目标不同，不相加为统一样本数；现场单帧置信度用于描述对应识别结果，不替代数据集准确率或混淆矩阵；混合统计来自既有原始记录，不使用未核准的板端墙钟日期推断实验时间。

3.3.2 连续整网日志（工程三）

连续整网归档共 83 条记录，硬件返回码全部为 0。日志中"推理段"的计时范围：从输入变换及保存之后开始，覆盖运行器装载、硬件执行、结果导出和解码；记录完成后才更新显示，因此不含输入变换和 OSD 更新。该范围内均值 11.016105 秒，中位数 11.023045 秒，最小 10.910632 秒，最大 11.127819 秒。

原始记录出现 Left、Right、Stop、ThumbsDown、ThumbsUp、Up 六类手势，未见 Down。当前结果用于说明六类已有日志和连续运行状态；七类完整板证仍需补充与该配置一致的 Down 原始记录及 HDMI 实拍。



图 3-3 连续整网 83 帧日志与计时范围

3.3.3 各形态时延与在线切换汇总

表 3-1 推理时延与在线切换汇总（各条目统计条件不同，不作跨条目排名）

实现/样本	指标	数值	说明
三任务物体检测	归档运行范围	约 0.9～1.2 秒	PS NanoDet COCO
三任务手势	归档参考	约 1.05 秒	PS YOLOv8n
三任务姿态	归档运行范围	约 2.9～3.9 秒	PS 拆分引擎

实现/样本	指标	数值	说明
固定输入 PL	九页计算/80MHz	4.267127 秒	较早稳定位流
连续 PL/83 条	日志推理段均值	11.016105 秒	不含输入变换与 OSD
混合/488 帧	日志中位数	2.496738 秒	另有 23 次恢复
双后端拨码	在线切换	11 次	纯 PS 与 PS+PL 帧边界切换



图 3-4 双后端不同手势的真实展示

3.3.4 各工程资源与时序

WNS（最差建立时间裕量）与 WHS（最差保持时间裕量）为时序约束检查结果，单位纳秒，正值表示满足约束。

表 3-2 各工程资源与时序（按各自实现报告统计）

工程与报告范围	LUT	FF	BRAM tile	DSP	WNS/ WHS ns
三网同任务/布局后资源、布线后时序	10652	15348	27.5	46	+6.126/+0.023

工程与报告范围	LUT	FF	BRAM tile	DSP	WNS/ WHS ns
三网三任务/布局后资源、布线后时序	14938	23429	67.5	17	+0.878/+0.009
纯 PL 整网/布线后	43504	38695	127	105	+0.099/+0.051
双后端拨码/布线后	31663	27748	96.5	91	+0.328/+0.023
多网换载/双后端基座参考	31663	27748	96.5	91	+0.328/+0.023

三网同任务与三网三任务分别对应 20.02%、28.08%的 LUT 占用，纯 PL 和双后端分别为 81.77%、59.52%。多网换载一行仅为当前设计基座，不代表在研终版资源或时序已经验收。WNS 不直接换算为系统最高频率；显示、摄像头与计算时钟按各自约束检查，所有已完成工程的上述配置均满足时序约束。

3.3.5 历史机械臂证据

图 3-5 汇总历史动作记录及独立实物素材。这些记录证明的是历史 PS 软件推理配置的机械臂闭环；纯 PL 整网形态因 PL 资源接近上限暂未集成该路径，其他当前链按实际配置独立验收。历史动作记录不延伸到新位流。



图 3-5 历史机械臂实物与独立动作台账

3.4 大模型协作与技能包

3.4.1 工作流与职责

项目使用 Claude Code、ChatGPT/Codex、Kimi 和 Zcode 四个平台的大模型完成方案讨论、代码辅助、资料定位和审计。协作流程：人工设定目标与边界→模型提出方案→真实工具执行→检查原始输出→纠错→分层验收→技能沉淀。实现会话与审计会话分开；人负责关键取舍与实物验收，模型生成的结论由对应日志和输出支撑。

已整理的四平台交互记录 V2 包含原始提示词、回答、失败观察和纠错链；逐字摘录与依据工程日志概括的案例分别标识，敏感凭据脱敏。本报告选取代表案例说明方法，完整记录随协作材料提供。

3.4.2 代表案例与自我纠错

表 3-3 大模型协作代表案例（原始记录摘要）

案例	提示与初始处理	失败观察与修正	验证及沉淀
空 lint 报告	要求仅用 Vivado 工具并与工具总数一致	解析器把帮助页脚误作两条违规；回查原始零违规总数	原始总数、CSV 行数、摘要一致
XSim 启动	检查陈旧仿真并结构化执行	启动失败；补齐子进程环境，分段检查	取得测试结束标记，连接与功能分级
OSD 乱码	修复标签显示并验证像素	原测试读取被测模块索引而漏错；改由规格独立推导	新 RTL 通过、旧 RTL 负控失败
部署竞争	更新驻留驱动与模型	在线覆盖导致进程读取不一致资产	先停链，再上传、校验、就绪检查
验收范围	审核"整网完成"结论	仅代表层通过且连续计数异常	限定结论范围，补完整输出和哈希

例一的原始要求包括"违规计数必须与 Vivado 报告的总数完全一致"，纠错要点是以工具原始统计为准；例二保留原始失败命令与最终分段执行日志，体现从错误到修复的轨迹。

3.4.3 AMD ROSS 使用经验

ROSS（AMD 的大模型工具编排系统）的主要价值在于把自然语言任务连接到可检查的 Vivado 会话、结构化操作和日志。项目中的直接收益是更容易定位仿真启动、会话归属、环境路径和 IP 例化问题；RTL 和测试正确性仍需结合实际例化模板、IP 配置和独立参考检查。

项目将官方工具能力与自定义规则配合：前者负责调用和读取状态，后者规定冻结工程边界、证据目录、版本哈希、失败回退及验收层级。对重复命令还需确认是否真正重跑，避免把缓存结果当作新实验。

3.4.4 技能包提炼及复用边界

表 3-4 技能包（从失败经验沉淀的可复用流程）

技能	来源问题	使用方式与已验证效果	适用及失效边界
PYNQ Overlay 工作流	下载成功但系统链未闭合	逐级检查 PS 兼容、复位、寄存器、DMA 和回退	跨板须改设备与地址
仿真工作流	子进程、库及顶层不匹配	检查依赖，分段编译展开运行	工具版本和 IP 库变化需重验
部署与哈希检查	运行中覆盖资产、版本串用	停链、成套上传、按映射比哈希	路径和服务需按项目适配
关键动作审计	代表性测试被扩大为整网结论	实现后独立回源，按输入与制品限定结论	证据缺失时保留未确认
OSD 负向对照	测试期望与被测模块同源	独立像素公式与已知坏版本验证测试有效性	换字体、缩放或编码需更新规格

技能包的价值在于其他队伍能够按适用范围、步骤、成功判据和失效条件迁移，而非复制本工程绝对路径。PYNQ 相关经验对应赛题鼓励的通用技能方向，跨板卡复用仍需按目标环境重新验证。

3.5 可复现性与交付说明

3.5.1 环境与版本准备

复现环境：Vivado 2025.2、目标器件 xc7z020clg484-1、匹配的 PYNQ 3.0.1 镜像及板端 ncnn 环境。开始前核对板型、启动介质、相机与 HDMI 连接、可用 DDR 和网络，并选择一个已完成工程配置。

首先校验包内 SHA-256 清单与实际文件，再检查工程文件、IP 配置、约束、源码和模型引用能否在目标环境解析。完整重建、基于现有位流重部署和读取归档结果属于三个不同层级；若重新生成位流，应保存新的哈希、资源与时序报告，并重新进行板端功能验证。

3.5.2 部署与运行步骤

第一步，确认模型、程序卷、BIT 和 HWH 来自同一工程包，记录输入与类别合同。第二步，停止旧摄像头服务和相关推理进程，确认 VDMA 停止及服务端口归属，再上传完整运行资产。第三步，依据本地到板端的文件映射逐项比对 SHA-256。第四步，在服务与硬件就绪检查后启动运行器，先用固定输入核对输出，再接入摄像头。第五步，观察模型选择、显示、输出序号、错误与恢复计数，并保存原始日志。

当前交付以既有环境重部署和归档结果复核为主；完整干净环境重建仍需按提交包逐项检查依赖。增量修复脚本不能替代首次部署入口；本地与远端文件名不同时应比较哈希摘要而非整行文本。

3.5.3 交付工程与复现入口

表 3-5 交付工程与复现入口

工程	板端或验证入口	必要资产	预期检查
三网同任务	relaunch_chain.sh	对应 BIT/HWH、三模型、注册表与链主	三模型就绪、统一预处理、手势结果与切换
三网三任务	launch_board.sh	对应 BIT/HWH、三任务模型、双服务端口	引擎就绪、三槽形状、三类 OSD 与切换
纯 PL 整网	launch_continuous.sh	BIT/HWH、程序卷、gold 参考、描述符和运行器	页完成、返回码、输入哈希与连续 JSONL
双后端切换	board_start_live_gesture7_dual.sh	双后端位流、两模型、六分支程序与双服务	后端就绪、拨码切换、检测显示与恢复计数
多网换载	当前为仿真验证入口	统一任务表、YOLOv8n/YOLO11n 资产与仿真台	仅检查阶段仿真结果，不作为成品复现入口

正式提交按 src、sim、build、board、data、skill 和 report 目录组织，并提供目录对照。每个工程同时保存运行脚本、模型与硬件版本、预期输出和对应验证记录，读者可按表 3-5 选择入口并核对结果。

第四部分 总结

4.1 可扩展之处

后续按“统一接口、换载闭环、编译提效、硬件扩展”推进。首先建立硬件能力合同与统一模型包，明确算子、形状、量化、表容量和地址约束；在已实现同 BIT 多网络多任务切换的基础上，完成 YOLO11n 仿真与板级联调，验证 YOLOv8n→YOLO11n→YOLOv8n 换载。其次构建图级与指令级两级 IR，自动生成权重布局、描述符、页表和调度程序，将固化计划面逐步改为可加载执行上下文。随后结合 Roofline、事件仿真和板上计数优化算子融合、数据复用与流水，按需扩展深度、分组卷积等算子，并以独立整数参考、XSim 和板测分层验收。

4.2 心得体会

本项目最重要的经验是先固定接口和验证方法，再追求性能。输入的颜色顺序、缩放、归一化和类别含义只要一项漂移，模型和硬件都可能正常执行却给出错误结果。把输入合同、量化规则、程序卷和位流绑定，使问题能够沿数据链逐层定位，也使不同推理形态的取舍更加清楚。

第二个经验是通用映射与单点性能需要分别评价。保留网络原始结构会增加资源受限平台上的优化难度，但也使纯 PL 整网积累能够复用到 PS+PL 子图推理。OPU 的价值不只是某个计算核，而是编译器、自定义指令集和硬件执行层共同形成的部署路径。后续提高速度，需要同时优化图编译、指令映射、流水和片上资源复用。

第三个经验是算术能力不等于整机速度。处理器数据整理、DDR 加载、缓存维护和运行器初始化都会消耗计算收益。固定输入实验的改进说明访存和调

度优化有效；混合模式仍慢于优化软件，则要求继续检查切点与跨端成本。资源优化同样需要正确的综合描述，合理的存储映射往往比盲目增加流水更有效。

第四点心得来自 AMD ROSS 套件的实际使用（工具组成见 3.4.3）。ROSS 把自然语言任务接到真实的 Vivado 会话和结构化工具上，我们的工作方式因此从"AI 会描述 Vivado 操作"变为"AI 能按规范实际执行 Vivado 操作"：仿真不再依赖预先打开图形界面，而是按 xvlog→xelab→xsim 分段调起，显式核对库名、顶层与终止标志——此前在编译子进程阶段反复失败的仿真启动，变成可稳定复现的流程；IP 例化一律以当前配置生成的.veo 模板逐端口核对，不凭模型记忆补写；工具命令与方法学问题先查 AMD 官方资料再动手，减少对模型记忆的依赖。同时保持三点清醒：相同命令短时间可能返回上一次结果，重跑必须显式区分轮次，不能把缓存当新实验；会话可用不代表设计正确，功能结果仍逐项验收；官方 Skill 提供通用方法，本工程再以自定义 Skill 固定调用授权、会话互斥、证据目录与分层验收，两者互补而非替代。ROSS 减少的是流程性错误和无意义返工，不是替人做工程判断——对同时推进 RTL、仿真、IP 与板级验证的项目，这种流程可靠性与可复现性就是最直接的收益。

大模型协作的整体心得与此一致：工程结论必须由真实输出支撑。保留失败记录、限定结论范围、设置独立审计并把经验写为可复用技能，使研发过程与最终结果同样具有可复查性。

第五部分 参考文献

[1] AMD. 全国大学生嵌入式芯片与系统设计竞赛 2026 选题指南 AMD[Z]. 2026: §3.3.
[2] AMD. Zynq-7000 SoC Data Sheet Overview DS190[EB/OL]. AMD Documentation.
[3] AMD. 7 Series DSP48E1 Slice User Guide UG479[EB/OL]. AMD Documentation.
[4] PYNQ Contributors. PYNQ Python Productivity for Zynq Documentation[EB/OL]. PYNQ, version 3.0.1.
[5] Tencent. ncnn A High-Performance Neural Network Inference Framework[CP/OL]. GitHub: Tencent/ncnn.
[6] AMD. Vivado Design Suite User Guide Synthesis UG901[EB/OL]. 2025.2.
[7] AMD. Vivado Design Suite User Guide Implementation UG904[EB/OL]. 2025.2.
[8] AMD. AXI Video Direct Memory Access Product Guide PG020[EB/OL]. AMD Documentation.
[9] Ultralytics. Ultralytics YOLO Documentation[EB/OL]. YOLOv8 and YOLO11 model documentation.
[10] UMUROGLU Y, FRASER N J, GAMBARDELLA G, et al. FINN: A Framework for Fast, Scalable Binarized Neural Network Inference[C]//Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. 2017: 65-74.
[11] NanoDet Contributors. NanoDet-Plus Super Fast and High Accuracy Lightweight Anchor-Free Object Detection Model[CP/OL]. GitHub: RangiLyu/nanodet.
[12] AMD. UltraFast Design Methodology Guide for FPGAs and SoCs UG949[EB/OL]. 2025.2.
[13] YU Y, WU C, ZHAO T, et al. OPU: An FPGA-Based Overlay Processor for Convolutional Neural Networks[J]. IEEE Transactions on Very Large Scale Integration (VLSI) Systems, 2020, 28(1): 35-47. DOI:10.1109/TVLSI.2019.2939726.
[14] 何磊, 王堃, 吴晨, 等. FPGA Overlay 处理器加速 AI 计算[J]. 中国科学：信息科学, 2025, 55(4): 796-836. DOI:10.1360/SSI-2024-0351.